E M B E D D E D C O D E A N A L Y S I S R E S E A R C H

嵌入式代码检测系统技术调研报告

检测对象 · 方法体系 · 多智能体协作框架选型

调研对象：嵌入式 C / C++ / Rust 代码检测 × 多智能体协作

调研方式：中英文公开资料检索与核实（120 余项来源）

报告版本：V1.0（调研阶段）

报告日期：2026 年 9 月 30 日

摘　要

本报告面向多智能体驱动的嵌入式代码检测系统的立项论证，回答三个核心问题：嵌入式代码检测的对象是什么（测什么）；检测方法应如何分类、各类有哪些可用开源工具（分为哪些方面）；有哪些效率较高、可直接复用的开源多智能体协作项目（用什么搭建）。调研于 2026 年 9 月完成，基于三个并行调研通道合计 120 余次中英文公开资料检索，框架星标数与工具版本均于 2026 年 9 月 30 日逐项核实。系统规划面向多种嵌入式语言（以 C 为主，兼顾 C++ 与 Rust），第一期覆盖静态缺陷检测、编码规范合规、并发与中断时序检测、单元测试生成与覆盖率四个维度。

关于测什么：检测对象建议按缺陷类型（对齐 CWE 分类）与规范条款（MISRA C、GJB 8114、CERT C）双维度组织检测项集，再按 ISO 26262、DO-178C 等行业标准要求裁剪。内存安全类缺陷是嵌入式场景中被实际利用最多的类别（2025 年 CWE Top 25 的 KEV 子榜中越界写得分 50.39，远超第二名 XSS 的 17.99），应作为最高优先级；并发与中断类缺陷是 RTOS 环境的特色高风险项，主流规范已相继补齐该维度。

关于怎么测：方法体系分为静态分析、动态分析、混合分析与二进制/固件分析四大族，四者按能查出与查不出什么的互补关系组合使用。开源工具链已具备以 SARIF 为统一告警格式的可编排能力，CBMC、gcovr、cppcheck 等关键工具均提供原生 JSON/SARIF 输出；但裸机 MCU 场景存在 ASan/TSan/Valgrind 不可用的根本限制，动态检测需走 CMock 打桩加 Renode/QEMU 仿真路线。

关于用什么搭建：多智能体框架推荐 LangGraph（生产可控、可观测性最好）与 AgentScope（异步并行、分布式、中文生态），代码审查功能可基于 PR-Agent 二开复用；AutoGen 已于 2026 年 4 月冻结，不应在新项目中采用。学界与工业界共识为：多智能体 token 成本约为普通对话的 15 倍，仅在可并行、广度优先的告警分诊与多视角审查场景收益明确。因此本系统推荐采用传统工具产出结构化告警、智能体并行研判、确定性流水线汇总裁决的混合架构，而非全自由多智能体协作。

目　录

摘　要 **1**

第一章　调研背景与范围 **2**

1.1 项目背景与动机 2

1.2 调研问题与方法 2

1.3 范围界定 2

第二章　检测对象：嵌入式代码测什么 **3**

2.1 缺陷类型全景：内存安全为第一优先级 3

2.2 编码规范与规则体系 3

2.3 功能安全标准对代码级验证的要求 4

2.4 固件与二进制层检测对象 4

第三章　检测方法分类与开源工具矩阵 **4**

3.1 四大方法族总览 4

3.2 静态分析与形式化方法工具 5

3.3 动态分析方法工具 5

3.4 仿真、追踪与固件分析工具 6

3.5 工具可编排性与统一告警格式 6

3.6 嵌入式动态测试的难点与解法 6

第四章　开源多智能体协作项目调研 **7**

4.1 通用多智能体框架横向对比 7

4.2 代码检测与审查类开源项目 7

4.3 静态分析与 LLM 结合的研究现状 8

4.4 效率结论：多智能体何时收益为正 8

第五章　推荐架构与选型建议.....8

5.1 总体架构：三层设计.....8

5.2 框架与模型选型.....9

5.3 分阶段落地路线.....9

5.4 风险与注意事项.....9

第六章　结论.....9

附录　参考来源.....10

注：本目录由域代码生成。文档编辑后如需刷新页码，请在目录上单击右键并选择“更新域”。

第一章　调研背景与范围

1.1 项目背景与动机

嵌入式软件已广泛部署于汽车、航空、医疗、工业控制等安全关键领域，其代码质量直接关联功能安全与信息安全。一方面，ISO 26262、DO-178C、IEC 61508、IEC 62304 等标准对代码级验证活动（静态分析、结构覆盖率、编码规范合规）提出了明确且随安全等级递增的要求，欧盟网络弹性法案（CRA）等法规进一步将固件级漏洞管理纳入强制范围；另一方面，传统检测工具链存在误报率高、报告可读性差、跨工具结果割裂、依赖专家经验解读等痛点，检测产能难以匹配代码规模的增长。

大语言模型（LLM）与多智能体技术的成熟为上述痛点提供了新的解法：利用 LLM 进行告警去误报与分级解释、自动生成单元测试、按标准条款生成合规报告等能力已被多篇研究与工业实践验证。本项目拟构建多智能体驱动的嵌入式代码检测系统，由传统工具负责精确检测、智能体负责语义研判与报告生成。本报告为立项前调研阶段的成果，用于支撑后续方案设计。

1.2 调研问题与方法

本次调研围绕三个问题展开：其一，嵌入式代码检测的对象与缺陷类型，即测什么；其二，检测方法的分类体系及各类代表工具，即分为哪些方面；其三，高效率开源多智能体协作项目盘点，即可复用的技术底座。调研采用三个并行通道（检测对象与标准、方法与工具、多智能体项目）进行中英文检索，合计 120 余次；关键事实（GitHub 星标数、工具版本与维护状态）于 2026 年 9 月 30 日通过 GitHub API 与官方网站核实，报告中涉及的数据均可溯源至附录所列来源。

1.3 范围界定

目标代码范围：面向多种嵌入式语言，以 C 语言（裸机与 RTOS 环境）为核心兼顾 C++ 与 Rust；运行环境覆盖裸机 MCU、RTOS（FreeRTOS、RT-Thread 等）与嵌入式 Linux。检测维度范围：第一期覆盖静态缺陷检测、编码规范合规（MISRA/GJB）、并发与中断时序检测、单元测试生成与覆盖率四项；固件二进制检测（无源码场景）作为扩展方向一并调研。本报告不涉及硬件在环台架选型与商业工具采购评估，商业工具仅作能力对照。

第二章　检测对象：嵌入式代码测什么

2.1 缺陷类型全景：内存安全为第一优先级

嵌入式代码的检测对象可归纳为七大类缺陷，优先级判断的直接依据是实际漏洞利用数据。在 MITRE 2025 年 CWE Top 25 及其 KEV（已知被利用漏洞）子榜中，越界写（CWE-787）KEV 得分 50.39，远超第二名 XSS 的 17.99，且 CWE-787 在 2024 版 CWE Top 25 中排名第一——内存安全类缺陷是嵌入式与物联网软件中被实际利用最多、危害最大的类别。表 2-1 给出七类缺陷及其嵌入式典型成因。

表 2-1 嵌入式代码典型缺陷类型与成因

缺陷类别	代表缺陷（对应 CWE 编号）	嵌入式典型成因
内存安全	缓冲区溢出（121/122）、数组越界（125/787）、空指针解引用（476）、释放后使用（416）、内存泄漏（401）、未初始化读取（457）	协议帧解析无界拷贝；无 MMU 平台越界写不崩溃而是静默破坏相邻任务内存；系统 7×24 长驻使小泄漏最终致命
整型与算术	有符号溢出（190）、无符号回绕（191）、隐式截断、除零（369）、移位越界	16 位 MCU 上 int 仅 16 位，隐式整型提升截断高发；ADC 采样值直接参与比例运算
未定义行为（UB）	严格别名违反、对齐违反、悬空指针等	GCC/IAR/Keil 对 UB 的优化方向不同，换编译器即暴露故障
并发与中断	数据竞争（362/366）、死锁（833）、优先级反转、ISR 可重入性、原子性缺失	ISR 与任务共享变量无保护；volatile 不保证原子性，多字节共享数据撕裂；RTOS 优先级反转有大量真实修复案例
栈与实时性	栈溢出、最坏执行时间（WCET）超时、阻塞调用	递归、大局部数组、中断嵌套；运行时检测属事后尽力而为，认证场景需静态最坏栈深证明
字符串与资源	不安全字符串函数（120/134）、格式化字符串（134）、外设句柄泄漏（404）、动态内存滥用	错误分支漏关外设句柄耗尽句柄池；安全标准普遍禁止或严格限制运行期动态内存分配
结构与质量	死代码、控制/数据耦合、圈复杂度超标、递归、多出口函数	DO-178C 明确的代码验证目标；影响可维护性与认证追溯

七类中有两类需特别说明。其一，内存安全类在无 MMU 平台的表现与通用软件差异显著：越界写不会触发段错误，而是静默破坏相邻任务的数据与栈，缺陷往往在远端现场才暴露，这使静态防线的价值在该场景显著高于通用软件。其二，并发与中断类缺陷在 RTOS 环境高发：ISO 26262-6:2018 已在编码指南主题中新增并

发（concurrency）主题，MISRA C:2023 的 AMD4 修订新增多线程与原子类型规则，主流规范均在补齐该维度，检测系统应将其列为独立检测项集。

2.2 编码规范与规则体系

编码规范将缺陷类型沉淀为可逐条检查的规则，是规范合规维度的检测对象基础。表 2-2 汇总主流规范的规模与工具支持现状。需要注意两点：一是 MISRA C:2025 已于 2025 年发布（新增 4 条规则，有效指南约 225 条），但开源工具尚无完整覆盖——cppcheck 开源版的 misra.py 插件覆盖 MISRA C:2012（规则文本需自备），MISRA C:2025 全覆盖目前仅在 Cppcheck Premium 25.8 商业版提供；二是国内军品场景以 GJB 8114-2013（124 条强制规则）与 GJB 5369-2005（138 条）为事实基线，两者规则编号与分类互不兼容，检测系统的规则库需按目标标准分集管理。

表 2-2 主流编码规范与规则规模

规范	适用语言	规模与要点	工具支持现状
MISRA C:2012（含 AMD1-4）	C	约 200 条指南；AMD4（2023）新增多线程与原子类型规则	cppcheck misra.py、clang-tidy、PC-lint Plus
MISRA C:2023	C	整合 2012 全部修订，覆盖 C90/C99/C11/C18	主流商业工具覆盖，开源按 2012 映射
MISRA C:2025	C	新增 4 条、删除 2 条，有效指南约 225 条	开源无全覆盖；Cppcheck Premium 25.8 全覆盖
MISRA C++:2023	C++	179 条指南（基于 C++17），在规则层面实际取代 AUTOSAR C++14	商业工具为主
SEI CERT C	C	98 条规则，按 PRE/DCL/INT/MEM/CON 等章节组织，与 CWE 有官方映射	cppcheck/clang-tidy 部分覆盖
GJB 8114-2013 / GJB 5369-2005	C/C++	124 条强制规则 / 138 条规则，国内军品静态检测事实基线	国产工具与自建规则库支持

2.3 功能安全标准对代码级验证的要求

行业标准不直接定义缺陷清单，而是规定必须执行的验证活动及其推荐强度，是检测项集裁剪的依据。表 2-3 汇总各标准要求。共性规律有三：静态分析的推荐强度随安全等级递增（ISO 26262 ASIL D 为最高级++，IEC 61508 高 SIL 等级与 EN 50716 全等级为 HR 即高度推荐）；结构覆盖率要求随等级从语句覆盖升至 MC/DC

（DO-178C DAL A 强制）；静态分析不能替代动态测试（DO-178C 明确）。值得关注的是，开源链路在 2024 至 2025 年完成关键闭环：GCC 14 原生支持 MC/DC 条件覆盖插桩，lcov 2.x 支持 MC/DC 报告，llvm-cov 支持相应编译选项，安全关键覆盖指标首次可用纯开源工具链度量。

表 2-3 功能安全与法规标准对代码级检测的要求

标准/法规	领域	代码级要求要点
ISO 26262-6:2018	汽车功能安全	静态分析在 ASIL D 为++；覆盖率随 ASIL 由语句升至 MC/DC；2018 版新增并发主题；设计原则限制动态内存
DO-178C	民用航空	DAL A 强制 MC/DC 覆盖率；死代码与控制/数据耦合分析；覆盖率须源于需求测试；静态分析不可替代动态测试
IEC 61508-3:2010	工业功能安全	编码标准在 SIL 3-4 为 HR；静态分析推荐级随 SIL 递增；附录 B 表化规定静态技术
EN 50716:2023	轨道交通（取代 EN 50128）	静态分析 SIL 1-4 均为 HR；SIL 4 期望 MC/DC；验证工具按 T1/T2/T3 分级鉴定
IEC 62304 + FDA 2026 指南	医疗器械	B/C 类强制单元实现验证；漏洞扫描、SCA（SBOM）、静态与动态分析列为上市前必备测试
EU CRA	欧盟网络安全法规	SBOM 须机器可读且覆盖固件及依赖；漏洞 24 小时预警、72 小时详报（2026-09 起生效）

2.4 固件与二进制层检测对象

当第三方组件无源码或需验证交付固件时，检测对象扩展到二进制层，包括：组件清单（SBOM）生成与已知 CVE 匹配、硬编码密钥与默认口令、暴露的调试接口（UART/JTAG）、过时组件与弱配置。法规侧，欧盟 CRA 自 2026 年 9 月 11 日起生效漏洞报告义务，并要求 SBOM 机器可读且覆盖固件；FDA 2026 年指南将固件安全测试列为医疗器械上市前必备项。固件层检测建议作为系统二期扩展方向，其技术路线见第三章第四节。

第三章　检测方法分类与开源工具矩阵

3.1 四大方法族总览

嵌入式代码检测方法分为四大族：静态分析、动态分析、混合分析与二进制/固件分析。方法选择的依据是互补性：静态分析覆盖面广但存在误报、需人工分诊；动态分析低误报但受测试激励质量限制；形式化方法可给出数学证明但算力成本高；固件分析面向无源码场景。表 3-1 给出四族对比。

表 3-1 四大方法族对比

方法族	原理	能查出	主要局限	计算成本
静态分析	规则/AST 模式匹配、编译器警告、数据流与控制流分析、抽象解释、符号执行、有界模型检查	内存/整型/UB 类缺陷、规范违规、部分并发问题	查不出真实时序与硬件行为；误报需分诊	规则检查秒级；形式化对大程序可达小时级
动态分析	单元测试加 mock、运行时插桩、覆盖率统计、运行时追踪	真实触发缺陷（低误报）、覆盖率证据、实时性与调度问题	未执行路径不可见；依赖激励质量	中等；需可执行环境与激励
混合分析（concolic）	符号执行与具体执行交替，SMT 求解生成新输入	深层路径缺陷；自动生成高覆盖测试用例	指针与外设寄存器建模困难；路径爆炸	高；SMT 求解为瓶颈
二进制/固件分析	解包提取、反汇编反编译、SBOM 与 CVE 匹配、固件仿真	无源码组件漏洞、硬编码密钥、过时组件	高度定制 MCU 难提取难仿真；编译期信息丢失	中等；全系统仿真成功率是短板

3.2 静态分析与形式化方法工具

静态工具按能力递进分三档。第一档为规则与数据流检查（cppcheck、clang-tidy、GCC -fanalyzer），秒级完成、适合 CI 常驻；第二档为路径敏感分析（Clang Static Analyzer、GCC -fanalyzer），可追踪缺陷传播路径；第三档为形式化方法：Frama-C 以抽象解释对运行时错误给出无误报界的证明，CBMC/ESBMC 以有界模型检查对断言、溢出、指针安全给出反例。交叉编译场景下 arm-none-eabi-gcc 自带 -fanalyzer，无额外部署成本。表 3-2 为工具矩阵，其维护状态与版本均于 2026 年 9 月核实。

表 3-2 静态分析与形式化开源工具矩阵

工具	原理定位	许可证	维护状态	结构化输出
cppcheck 2.22	规则加数据流，专注未定义行为；官方 MISRA 插件	GPL-3.0	活跃（2025-09 发布 2.22）	XML；SARIF（2.15 起）
clang-tidy	AST 级规则检查加自动修复，需编译数据库	Apache-2.0（LLVM）	随 LLVM 持续发布	YAML；配 CodeChecker 出 JSON
Clang Static Analyzer	路径敏感符号化流分析	Apache-2.0	随 LLVM 持续发布	plist（机器可读）
GCC -fanalyzer	编译器内建路径敏感静态分析器	GPL-3.0	GCC 14 起误报显著下降	JSON/SARIF（GCC 13 起）
Frama-C 33	抽象解释（Eva）加 ACSL 契约演绎验证	LGPL-2.1	活跃	需脚本封装
CBMC 6.11	有界模型检查（SAT/SMT）	BSD 类	活跃（2025-08）	原生 JSON（--json-ui）
ESBMC 7.x	SMT 有界模型检查加并发调度与偏序约简	宽松 BSD	活跃（SV-COMP 常年参赛）	JSON 接口
KLEE 3.1	LLVM 动态符号执行，自动产出测试用例	类 BSD	维护中	ktest 格式，需封装
angr 9.2	二进制符号执行与分析框架	BSD	活跃	Python 库，天然可序列化

3.3 动态分析方法工具

动态检测存在环境分层的根本限制。 AddressSanitizer、ThreadSanitizer、MemorySanitizer 与 Valgrind 均依赖宿主环境（嵌入式 Linux 或 64 位平台），裸机 MCU 上仅 UndefinedBehaviorSanitizer 提供可裁剪的 standalone 运行时。因此裸机动态检测的主路线是：宿主机上以 CMock 对硬件抽象层打桩、运行 Unity 单元测试（Ceedling 一键串联构建与覆盖率），需要外设行为时引入 Renode 仿真。表 3-3 为工具矩阵。

表 3-3 动态分析开源工具矩阵

工具	原理定位	许可证	维护状态	结构化输出
Unity / CMock / Ceedling	C 单元测试、自动 mock 与构建编排，嵌入式事实标准三件套	MIT	活跃（Ceedling 1.0）	JUnit XML

工具	原理定位	许可证	维护状态	结构化输出
GoogleTest	C++ 测试框架，嵌入式 Linux 侧常用	BSD-3	活跃	原生 JSON/JUnit
Criterion	现代 C/C++ 测试框架	MIT	低频维护	JSON/JUnit/ TAP
gcov / gcovr 8.6	覆盖率统计与跨编译器汇总，GCC 14 起支持 MC/DC	GPL-3.0 / BSD-3	活跃（2026-01 发布 8.6）	原生 JSON 与 Cobertura XML
lcov 2.3	覆盖率前端，2.x 支持 MC/DC 报告	GPL-2.0+	活跃	.info 格式，需转换
ASan / TSan / MSan	运行时内存错误、数据竞争、未初始化检测	Apache-2.0 / BSD	随编译器演进	日志输出；仅嵌入式 Linux 可用
UBSan	未定义行为运行时检测，standalone 版裸机可用	同上	随编译器演进	日志输出
Valgrind 3.27	影子内存全量检测，无需重编译	GPLv3	活跃	原生 XML（--xml=yes）；仅 Linux

3.4 仿真、追踪与固件分析工具

全系统仿真与追踪工具解决动态检测的运行环境问题，固件分析工具面向无源码场景。表 3-4 汇总该组工具。其中 Renode 对 MCU 与多节点系统的外设级仿真能力与 Robot Framework 测试集成使其成为仿真中动态测试的最佳开源载体；固件自动化安全分析首选 EMBA（内置 SBOM 检测与 CVE 匹配），binwalk v3 上游维护已转弱，解包环节可由新兴替代品 unblob 补充。

表 3-4 仿真、追踪与固件分析开源工具

工具	原理定位	许可证	维护状态	结构化输出
Renode	MCU/SoC 多节点外设级仿真，支持故障注入与 CI	MIT	活跃	Robot Framework XML
QEMU 10.x	全系统仿真（嵌入式 Linux 侧动态测试载体）	GPL-2.0	活跃	QMP（JSON 控制接口）
SEGGER SystemView	RTOS 事件级追踪与时间线分析	非商业免费	持续维护	录制文件，面向人工分析

工具	原理定位	许可证	维护状态	结构化输出
EMBA v2	固件自动化安全分析加 SBOM 与 CVE 匹配	AGPL-3.0	活跃	HTML/日志，解析需自封装
Ghidra 12.x	反汇编反编译，支持 headless 与脚本	Apache-2.0	活跃	脚本自定义
binwalk v3	固件镜像签名扫描与文件系统提取	MIT	上游放缓（社区分叉接续）	文本日志

3.5 工具可编排性与统一告警格式

多智能体系统将上述工具作为可调用工具编排，前提是工具具备命令行接口与结构化输出，这是选型的硬约束。按输出能力可分三档：一等（原生 JSON/XML/SARIF，可直接被智能体消费）：CBMC（--json-ui）、ESBMC、gcovr（--json）、cppcheck（XML/SARIF）、GoogleTest 与 Criterion（JSON/JUnit）、Ceedling（JUnit XML）、Valgrind（--xml）、Renode（Robot XML）；二等（结构化但需封装）：Frama-C、KLEE、lcov、Ghidra headless；三等（面向人读）：SystemView、Tracealyzer、EMBA。建议以 SARIF（OASIS 静态分析结果交换标准）作为统一告警中间格式：cppcheck 2.15 起、GCC 13 起、Clang 均已原生支持，各工具告警归一为 SARIF 后进入智能体研判层，可显著降低适配层复杂度。

3.6 嵌入式动态测试的难点与解法

嵌入式动态测试存在四大难点：硬件依赖（无真实外设则相关路径不可触发）、传感器激励难以构造（电压电流时序上逼真的模拟量难以人工生成）、时序不可复现（中断、DMA 与多任务并发使缺陷复现非确定）、目标板观测性差（插桩与覆盖率统计开销大）。常用解法按层级划分见表 3-5，其核心思想是把不可控的真实环境逐层替换为可脚本化、可注入、可重放的受控环境。

表 3-5 嵌入式动态测试难点与分层解法

层级	手段	代表开源工具
单元层（宿主机）	硬件抽象层打桩、外设寄存器映射为内存结构体、测试驱动开发	CMock + Unity + Ceedling
进程级（嵌入式 Linux）	运行时插桩检测与覆盖率统计	Sanitizers、Valgrind、gcov/gcovr

层级	手段	代表开源工具
全系统仿真	外设级仿真替代真实硬件，故障可注入、时序可编排、CI 可复现	Renode、QEMU
硬件在环（HIL）	实时仿真器模拟传感器与总线，支持故障组合遍历与再现	商业台架为主；开源以 Renode 加自研信号板折中
时序追踪	RTOS 事件追踪与时间线分析，定位时序缺陷	SEGGER SystemView、Tracealyzer

第四章　开源多智能体协作项目调研

4.1 通用多智能体框架横向对比

截至 2026 年 9 月 30 日对十个主流框架的核实结果见表 4-1 与图 4-1，可得出三点判断。其一，AutoGen（61,236 星）已于 2026 年 4 月冻结归档，官方路线并入 Microsoft Agent Framework，社区延续为 AG2（约 5,000 星），新项目不应再采用 AutoGen；其二，框架层正快速收敛（AutoGen 与 Semantic Kernel 合并、Swarm 升级为 OpenAI Agents SDK），通用编排抽象本身在贬值，工程价值转移到上下文工程与评测体系；其三，对本系统而言检测流程确定、要求可审计与人机协同，图/状态机范式的 LangGraph 最匹配，若需多项目并发横向扩展则 AgentScope 的异步并行与分布式调度更优。

表 4-1 通用多智能体框架横向对比（Star 数为 2026-09-30 实测）

框架	架构模式	GitHub Star	状态与许可证	选型建议
LangGraph	图/状态机，checkpoint 持久化	42,507	活跃；MIT	生产首选：可控性、可观测性、人机协同最好
CrewAI	角色分工 Crew/Task/Flow	59,215	活跃；MIT	原型最快；细粒度控制弱
AgentScope 1.0	ReAct 加异步并行与分布式	32,613	活跃；Apache-2.0	分布式扩展与中文生态优选
AutoGen	对话群聊多智能体	61,236	已冻结（2026-04）；CC-BY-4.0	不建议新项目采用
Microsoft Agent Framework	AutoGen 与 SK 合并的企业框架	13,877	活跃；MIT	微软生态企业路线
MetaGPT	SOP 角色流（软件公司模拟）	70,701	更新放缓；MIT	研究参考，借鉴其 SOP 角色结构
OpenAI Agents SDK	轻量 handoff 加 guardrails	29,783	活跃；MIT	抽象最少，适合嵌入自有系统
smolagents	Code Agent（写代码即动作，省 token）	29,605	活跃；Apache-2.0	极轻量内嵌方案
OpenHands	事件流编码 Agent 平台（含沙箱）	89,593	极活跃；MIT	可整体二开为检测平台

框架	架构模式	GitHub Star	状态与许可证	选型建议
Dify	低代码 Agent 平台	157,566	极活跃；OSS 许可证对多租户商用有限制	平台型方案，商用需审查条款

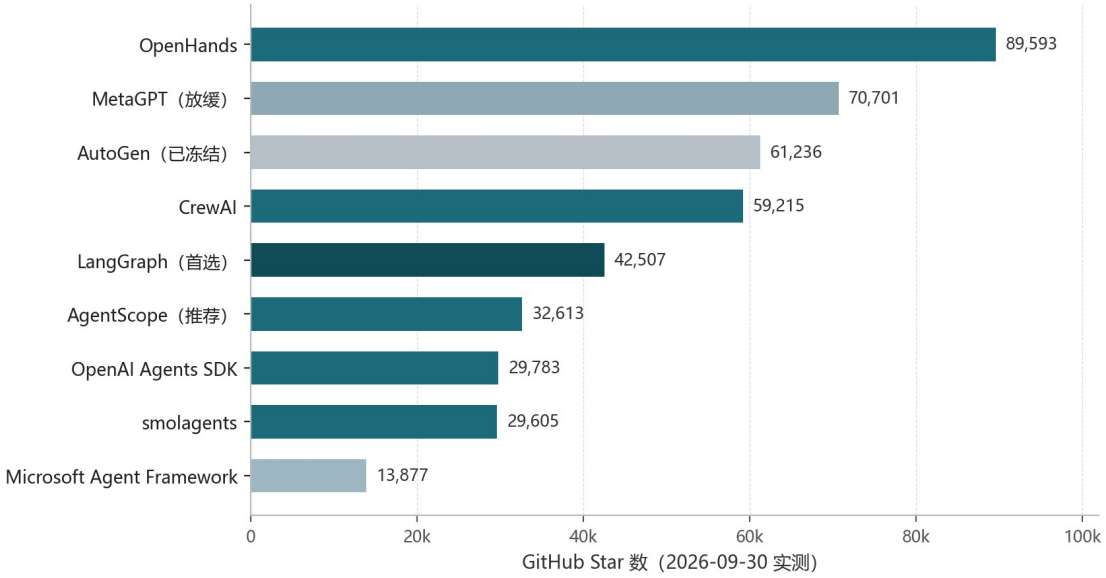


图 4-1 主流多智能体框架 GitHub Star 数对比（2026-09-30 实测，未含 Dify 157,566）

4.2 代码检测与审查类开源项目

SWE-bench 系榜单项目证明了架构选择对成本的影响可达一个数量级。Agentless 以三阶段简单流水线（分层定位、修复、验证）取得 SWE-bench Lite 27.33%，单实例成本 0.34 美元，约为同期多智能体方案的十分之一；mini-SWE-agent 以约 100 行 bash 循环达到 Verified 59.8%、单实例 0.04 美元，为当前性价比天花板。代码审查方向，PR-Agent（13,199 星，MIT）是最成熟的自托管开源方案，支持任意 LLM 与主流代码托管平台，可直接作为审查功能的二开基座。表 4-2 汇总可直接复用的项目与要点。

表 4-2 代码检测与审查类开源项目清单

项目	定位	关键数据	可复用点
Agentless	三阶段非智能体流水线（定位-修复-验证）	SWE-bench Lite 27.33%，0.34 美元/实例（约为同期 agent 方案 1/10）	分层定位加语法树上下文加复测验证的低成本范式
mini-SWE-agent	约 100 行 bash 循环的极简智能体	Verified 59.8%，0.04 美元/实例	极简脚手架与智能体-计算机接口设计原则

项目	定位	关键数据	可复用点
OpenHands	事件流编码 Agent 平台，含沙箱运行时	Verified 53%，0.69-1.13 美元/解决实例	平台化二开为代码体检与修复服务
PR-Agent	开源 PR 审查工具集（review/describe/improve）	13,199 星，MIT，任意 LLM 自托管	代码审查功能直接二开基座
MARS（论文）	多智能体评审：发起人、评审委员会、主持人	角色拓扑效率优于自由辩论	评审委员会拓扑可在 LangGraph 上复现
ChatDev	多智能体软件公司模拟	34,425 星；ACL 2024	写码-审查-修复-测试 SOP 模板
CodeR（论文）	任务图引导的多 Agent 修复流水线	SWE-bench Lite 28.33%	任务图引导上下文检索的分工结构
FirmAgent（NDSS 论文）	模糊测试与 LLM 混合的固件漏洞发现	以 fuzzing 证据约束 LLM 幻觉	嵌入式固件方向的技术路线参考

4.3 静态分析与 LLM 结合的研究现状

与本系统形态最对口的方向是静态告警去误报。多篇 2024 至 2026 年研究（如 arXiv 2411.03079）验证了以精确代码上下文引导 LLM 判定静态分析误报的可行性；CMU SEI 的评估同时提醒，LLM 判警自身会引入新的误报与漏报，需人机协同兜底嵌入式方向，2025 年实测表明主流 LLM 无法直接生成完全 MISRA 合规的代码，而以静态分析器反馈闭环引导生成可使违规减少约 83%——这印证了本系统以工具检测为主、LLM 研判为辅的定位。固件方向已有 EMBA 加 Ghidra 加 LLM 的流水线实践与 FirmAgent 等混合方案。目前该方向尚无开箱即用的成熟开源项目，多为论文原型，恰是自研的空间所在。

4.4 效率结论：多智能体何时收益为正

关于多智能体与简单流水线的效率之争，2024 至 2026 年已形成三条硬数据与明确边界。硬数据：Anthropic 官方披露多智能体系统 token 消耗约为普通对话的 15 倍；2026 年研究（arXiv 2604.02460）证明归一化测试算力后单智能体可与多智能体打平，即多智能体报告的增益大部分来自更多算力而非协作本身；MAST（NeurIPS 2025）标注出多智能体系统三大失败类——规范缺陷、智能体间失调、验证缺失。收益边界：任务可并行、广度优先、单点失败可容忍的场景（全库扫描、逐条告警

分诊、多视角审查）收益明确；强顺序依赖的长链路任务、无验证环节的讨论则是负收益。对本系统的直接推论：告警分诊采用并行扇出（每条告警独立研判，天然可并行且失败可容忍），汇总与报告采用确定性流水线，避免自由辩论式协作。综合判断，代码检测恰好是多智能体收益明确的任务形态，但正确架构是编排确定的工作流加定向并行的分诊智能体，而非通用自治多智能体。

第五章　推荐架构与选型建议

5.1 总体架构：三层设计

推荐三层架构。工具执行层：以调度器封装 cppcheck、clang-tidy、GCC -fanalyzer、CBMC、Ceedling 等命令行工具，输出统一归一为 SARIF/JSON 告警池，工具调用与结果落盘全程可追溯。智能体研判层：并行分诊智能体对每条告警独立执行误报判定、严重性分级、根因解释与修复建议（结论附代码证据链），对高危项可选启用评审委员会式多智能体交叉审查（参考 MARS 角色拓扑）。报告与合规层：按标准条款映射（MISRA/GJB/ISO 26262）生成合规报告与度量看板，高危项经人工复核（利用 LangGraph 的 interrupt 人机协同机制）后关闭。该架构的本质是让传统工具负责精确与穷尽，让智能体负责语义理解与表达，让确定性流水线负责汇总与一致性。

5.2 框架与模型选型

编排框架首选 LangGraph（42,507 星，MIT）：图/状态机范式与检测流水线天然匹配，checkpoint 支持断点续跑，interrupt 支持人机协同，配合 LangSmith 可获得全链路可观测性。备选 AgentScope 1.0（32,613 星，Apache-2.0），其异步并行与分布式调度适合多项目并发横向扩展，中文生态好。原型验证期可用 CrewAI 快速搭建角色化流水线。不建议采用已冻结的 AutoGen。代码审查功能可基于 PR-Agent 二开。模型侧：Qwen3-Coder、GLM、DeepSeek 等国产模型在 BFCL 工具调用榜单上已与闭源前沿模型打平，自建系统无模型可得性障碍；建议按任务分级调用——海量告警分诊用轻量模型、报告生成与复杂研判用强模型，以控制 15 倍成本乘数的影响。

5.3 分阶段落地路线

结合第一期四维度的优先级与依赖关系，建议按四个阶段推进，每阶段以可演示产出收口，见表 5-1。

表 5-1 分阶段落地路线

阶段	目标	核心内容	里程碑产出
一（MVP）	静态缺陷检测加告警分诊端到端跑通	cppcheck/clang-tidy/-fanalyzer 工具适配器、SARIF 告警池、并行分诊智能体、检测报告 v1	对开源嵌入式工程的端到端检测报告

阶段	目标	核心内容	里程碑产出
二	编码规范合规检测	MISRA C:2012 与 GJB 8114 规则库、条款映射、合规报告模板	合规检测报告样例
三	并发时序检测与测试生成	最坏栈深分析接入、Ceedling 测试自动生成、覆盖率度量闭环	测试生成与覆盖率闭环演示
四	扩展与规模化	固件层检测（EMBA 集成）、多项目并发（分布式部署）、评审委员会交叉审查	规模化部署与评测报告

5.4 风险与注意事项

四项主要风险与对策：（1）LLM 幻觉与判警偏差——研判结论必须附代码证据链，高危项强制人工复核，并参照 c-CRAB 基准的二次验证方法建立分诊质量评测；（2）token 成本——多智能体约 15 倍的成本乘数要求按告警严重性分层路由，低危项用轻量模型处理，并设定单项目成本上限；（3）评测基准缺失——嵌入式检测无公开基准，需自建含注入缺陷的基准项目集，覆盖四维度各缺陷类型；（4）许可证合规——EMBA 为 AGPL-3.0，Dify 的 OSS 许可证对多租户商用有限制，集成前需逐项审查。另需注意 cppcheck 开源版 MISRA 插件不含 MISRA C:2025，规则库须按目标标准版本管理，避免合规声明失实。

第六章　结论

对三个调研问题的最终回答如下。测什么：以七类缺陷（对齐 CWE）与规范条款（MISRA C、GJB 8114、CERT C）双维度组织检测项集，按 ISO 26262、DO-178C 等标准裁剪，内存安全为第一优先级，并发与中断为嵌入式特色重点。分为哪些方面：静态分析、动态分析、混合分析、二进制/固件分析四大族，开源工具链（cppcheck、clang-tidy、GCC -fanalyzer、Frama-C、CBMC、Unity/CMock/Ceedling、gcovr、Renode、EMBA 等）已具备以 SARIF 为统一格式的可编排能力，裸机动态检测走打桩加仿真路线。用什么搭建：编排框架选 LangGraph（生产）或 AgentScope（分布式），审查功能参考 PR-Agent，告警分诊采用并行扇出智能体，整体为工具层产出结构化告警、智能体并行研判、确定性流水线汇总的三层混合架构。

下一步建议进入方案设计阶段，输出系统详细设计（智能体拓扑、工具适配器接口、SARIF 数据模型、评测方案），并选取一个开源嵌入式工程完成端到端原型验证，以实测数据校准本报告的选型判断。

附录　参考来源

以下为本报告关键结论的来源清单，按主题分组；星标数与版本信息核实时间为 2026 年 9 月 30 日。

一、标准与规范

MISRA 官方出版物：misra.org.uk/publications

Parasoft：MISRA C:2025 研讨会与变更解读（parasoft.com/webinar/misra-c-2025）

Qt Group：MISRA C:2025 变更解读（qt.io/software-insights）

AUTOSAR C++14 指南 R22-11（autosar.org）

SEI CERT C 编码标准（sei.cmu.edu/library/sei-cert-c-and-c-coding-standards）

MITRE 2025 CWE Top 25 与 KEV 子榜（cwe.mitre.org/top25/archive/2025）

ISO 26262-6:2018 方法表映射（public-docs.ferrocene.dev）

LDRA：IEC 61508-3:2010 实施白皮书（ldra.com）

Verifysoft / QA Systems：EN 50128 与 EN 50716 合规（verifysoft.com、qa-systems.com）

Finite State：EU CRA 与 SBOM 技术要求（finitestate.io/blog）

二、检测工具

cppcheck 发布记录（github.com/cppcheck-opensource/cppcheck/releases）

CBMC（cprover.org/cbmc）与 ESBMC（github.com/esbmc/esbmc）

Frama-C 下载与版本（frama-c.com）

KLEE v3.1（klee-se.org）；angr（angr.io）

GCC 14 静态分析改进（developers.redhat.com）与 GCC MC/DC 补丁（gcc.gnu.org）

Valgrind 新闻（valgrind.org/docs/manual/dist.news.html）

gcovr 变更记录（gcovr.com）；lcov MC/DC 支持（github.com/linux-test-project/lcov）

ThrowTheSwitch Ceedling/Unity/CMock（github.com/throwtheswitch）

Renode（antmicro.com/platforms/renode）；QEMU 10.0 发布（qemu.org）

EMBA（github.com/e-m-b-a/emba）；Ghidra 发布（github.com/NationalSecurityAgency/ghidra/releases）

SEGGER SystemView 与 Percepio Tracealyzer 授权说明（segger.com、percepio.com）

三、多智能体框架与项目

LangGraph（github.com/langchain-ai/langgraph）；CrewAI（github.com/crewAIInc/crewAI）

AgentScope 1.0（github.com/agentscope-ai/agentscope）与论文（arxiv.org/abs/2508.16279）

AutoGen 冻结公告与 Microsoft Agent Framework（github.com/microsoft/autogen）

OpenHands（github.com/All-Hands-AI/OpenHands）与 CodeAct 2.1 博客（openhands.dev）

mini-SWE-agent（github.com/SWE-agent/mini-swe-agent）；SWE-bench 榜单（swebench.com）

Agentless（github.com/OpenAutoCoder/Agentless）

PR-Agent（github.com/The-PR-Agent/pr-agent）；ChatDev（github.com/OpenBMB/ChatDev）

Dify（github.com/langgenius/dify）及 OSS 许可证条款

四、论文与研究

Agentless：arXiv 2407.01489；榜单可靠性解剖：arXiv 2506.17208；SWE-Effi：arXiv 2509.09853

MARS 多智能体评审：arXiv 2509.20357；More Agents Is All You Need：arXiv 2402.05120

SAST 误报消减：arXiv 2411.03079 与 arXiv 2601.18844；CMU SEI：LLM 评估静态分析告警（sei.cmu.edu/blog）

LLM 生成 MISRA 合规代码实测：arXiv 2506.23535；ACM：Towards MISRA-Compliant C Code Generation

MAST 多智能体失败研究：arXiv 2503.13657；算力归一对比：arXiv 2604.02460；AFlow：arXiv 2410.10762

Anthropic：多智能体研究系统构建（anthropic.com/engineering）

FirmAgent（NDSS）；EMBA+Ghidra+LLM 固件流水线（Preprints 202510.0166）；LLM4Decompile（github.com/albertan017/LLM4Decompile）